

CLASE 5

Completemos el diseño UML

Definiremos qué se puede ver, cómo se comunican los métodos y qué relaciones expresan pertenencia o especialización

Encapsular

Organizar

Justificar

Recordemos: un entrenador participa en relaciones diferentes



Se explicó quién se relaciona, cuántos participan y quién necesita conocer a quién.

¿Qué relación permanece y cuál existe solamente durante una operación?

**Entrenador-Inventario y Entrenador-Equipo permanecen;
intentar capturar una Criatura puede representarse como una dependencia temporal.**

CONCEPTOS POO: RESUMEN RÁPIDO

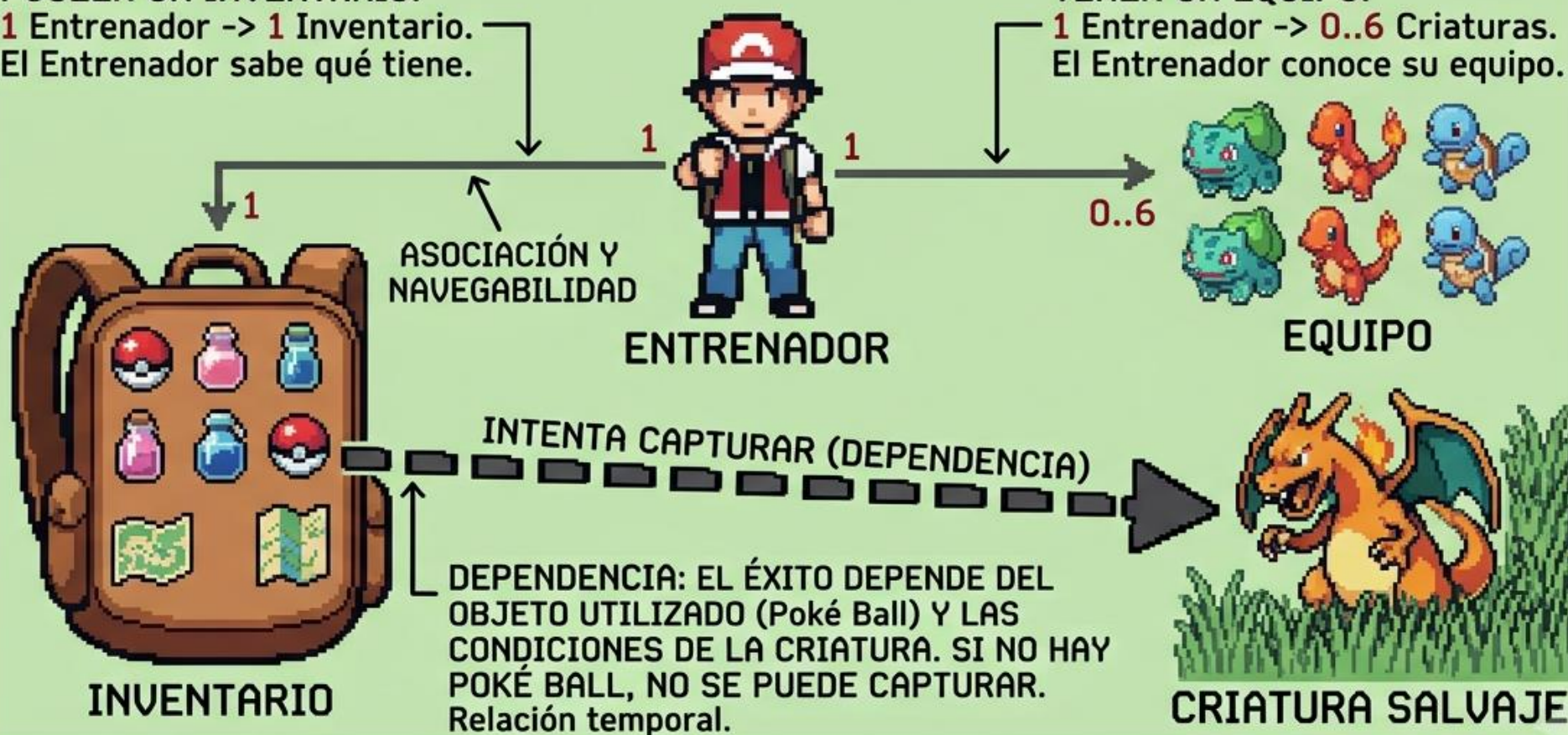
GAME BOY EDITION

POSEER UN INVENTARIO:

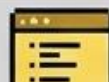
1 Entrenador -> 1 Inventario.
El Entrenador sabe qué tiene.

TENER UN EQUIPO:

1 Entrenador -> 0..6 Criaturas.
El Entrenador conoce su equipo.



ESTADO



MÉTODOS



NOTAS DE CLASE:

NO REPETIR LA CLASE COMPLETA

Hoy completaremos y defenderemos el UML de captura

OBJETIVO DE APRENDIZAJE

Representar visibilidad, tipos, parámetros, retornos y relaciones avanzadas para justificar el encapsulamiento y la organización de las clases del caso.

Al finalizar podrás...

- 01 Escribe atributos y métodos utilizando una notación UML legible.
- 02 Distingue asociación, agregación y composición mediante el ciclo de vida.
- 03 Utiliza herencia y polimorfismo solamente cuando existe una característica común justificable.

Entendiendo UML: Resolviendo la Ambigüedad entre Entradas y Salidas de Métodos

LO QUE EL MÉTODO NECESITA (ENTRADA)

- objetivo : Criatura
- cantidad : Integer
- probabilidad : Real



LO QUE EL MÉTODO ENTREGA (RETORNO)

- **Boolean**: confirma éxito o rechazo
- **Integer**: entrega una cantidad
- **void**: ejecuta una acción sin devolver

MÉTODO	DATOS DE ENTRADA	RETORNO DE VALOR	RESULTADO VISUAL
tieneEspacio() : Boolean	NINGUNO ✗	Boolean ✓ ✗	→ Gear → ✓ ✗
agregar(criatura : Criatura) : Boolean	criatura : Criatura 🐉	Boolean ✓ ✗	→ 🐉 → Gear → ✓ ✗
seleccionarEquipo() : void	NINGUNO ✗	void ✗	→ Gear CONFIRMACIÓN INTERNA

¿QUÉ ES `VOID`?

El tipo de retorno `void` **NO SIGNIFICA** que el método "no haga nada". Significa que el método realiza una acción (como actualizar el estado del equipo) pero **no devuelve ningún valor** al que lo invocó.



+ **agregar(criatura : Criatura) : Boolean** recibe una criatura y confirma si pudo incorporarla.

ANÁLISIS DE LA SINTAXIS DE UNA CLASE UML COMPLETA

Entrenador

```
- nombre : String
- nivel : Integer
# probabilidadBase : Real

+ intentarCaptura(objetivo : Criatura) : Boolean
+ seleccionarEquipo() : void
# calcularProbabilidad(base : Real) : Real
```

1. Leer de izquierda a derecha



2. **String, Integer, Real, Boolean:** Tipos conceptuales que luego se traducen a Python



3. **Parámetros en ():** Datos necesarios para ejecutar la operación. Se mantienen () vacíos si no hay parámetros



4. **Tipo de retorno:** Ubicado después del último signo dos puntos (:), indica el resultado del método



CONTROL DE ACCESO Y PROTECCIÓN DEL ESTADO DEL OBJETO

– Privado



Acceso Restringido.

Solo los métodos de la propia clase pueden acceder o modificar directamente este dato.

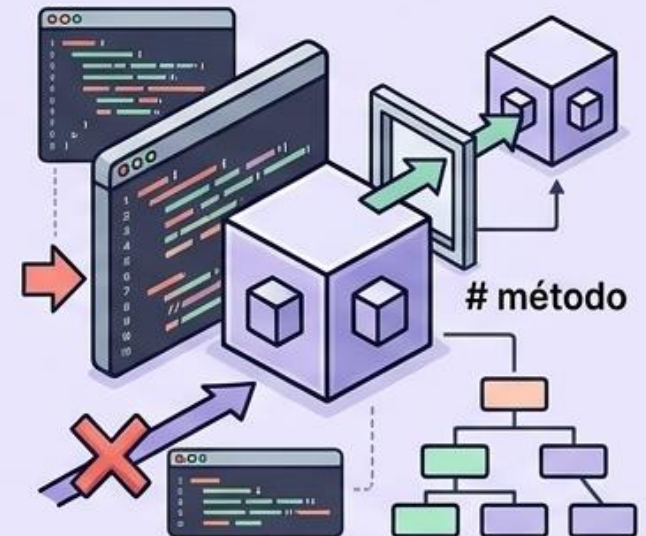
+ Público



Acceso Abierto.

Interfaces y métodos públicos disponibles para interactuar y colaborar con otras clases.

Protegido



Acceso Selectivo.

Accesible para la clase base y sus clases derivadas en la jerarquía de herencia.

EL ENCAPSULAMIENTO OCULTA EL ESTADO INTERNO Y EXPONE OPERACIONES CONTROLADAS QUE GARANTIZAN LA INTEGRIDAD DE LAS REGLAS DEL OBJETO.

DISTINGUIENDO RELACIONES DE PERTENENCIA

Una pregunta verificable sobre independencia y ciclo de vida.

LA PREGUNTA CLAVE PARA VERIFICAR: ¿Es la Parte independiente del Todo?

AGREGACIÓN ◇: Relación Débil

- Pertenencia Temporal.
- La Parte (Criatura) puede existir fuera del Todo (Equipo).
- Si el Todo (Equipo) desaparece (✗), la Parte (Criatura) sigue existiendo (✓).
- *Ejemplo de la Clase:* Una Criatura puede ir a la reserva o a otro equipo.



◇ Diamante Vacío (junto al Todo)

COMPOSICIÓN ◆: Relación Fuerte

- Pertenencia Exclusiva.
- La Parte (Inventario) depende del ciclo de vida del Todo (Entrenador).
- Si el Todo (Entrenador) desaparece (✗), la Parte (Inventario) también desaparece (✗).
- *Ejemplo de la Clase:* Para este modelo inicial, el Inventario se crea y elimina con el Entrenador.

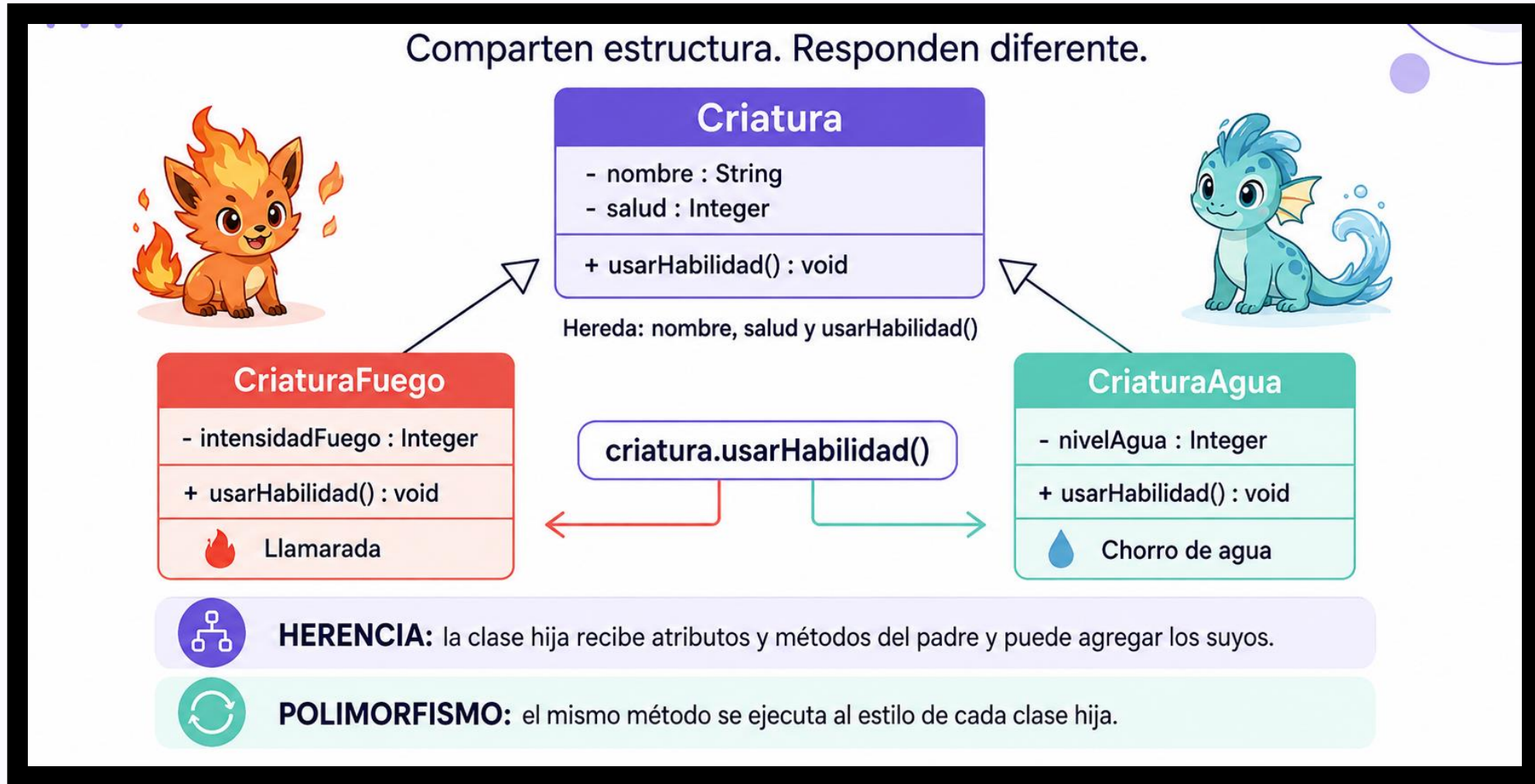


◆ Diamante Lleno (junto al Todo)

¿Cómo verificar? Con esta pregunta: "Si el Todo desaparece, ¿la Parte tiene sentido por sí misma?"

IMPORTANTE: Estas son *decisiones del modelo* respaldadas por reglas. Una regla diferente (ej: "El inventario se subasta si el Entrenador muere") **cambiaría** la respuesta a Agregación.

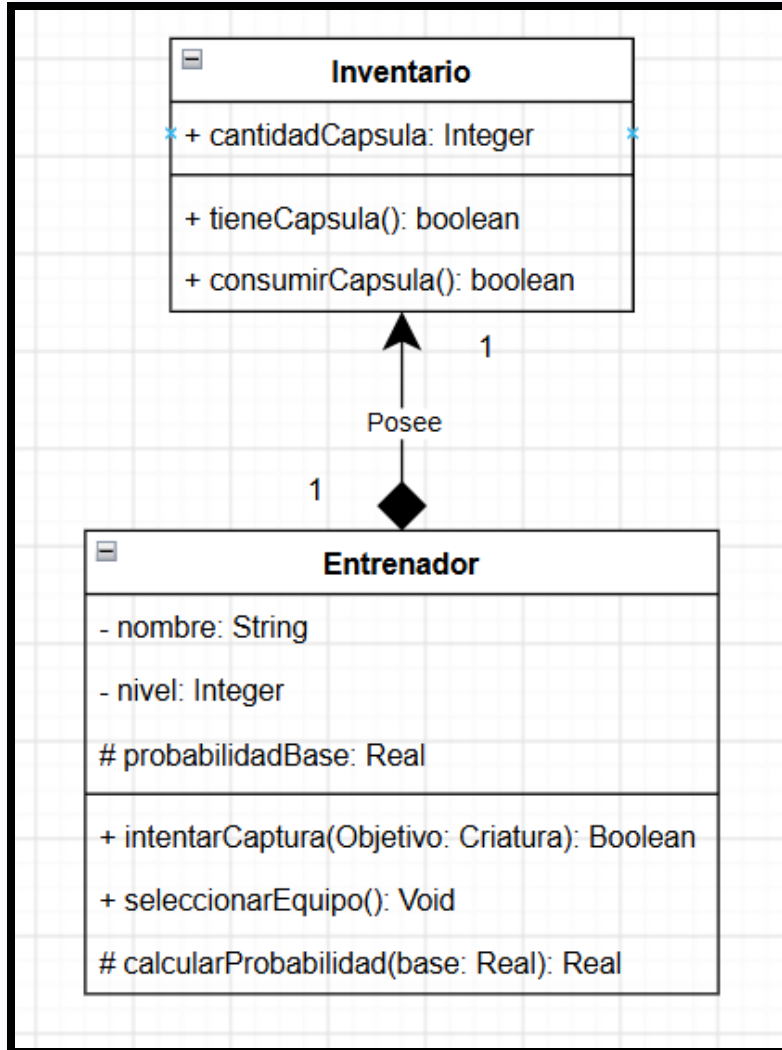
La herencia comparte una idea; el polimorfismo cambia su respuesta



La flecha de triángulo vacío apunta desde la clase especializada hacia la clase general.

Actividad

Transforma el UML v1 en una decisión defendible



Observa

- El dato queda privado para evitar modificaciones sin validación.
- Los **métodos públicos** controlan la consulta y el consumo.
- El diamante lleno queda junto a Entrenador, considerado el todo.
- La **composición** se acepta porque el inventario pertenece a ese entrenador durante su ciclo de vida.
- **Navegabilidad:** desde Entrenador se puede acceder a Inventario.

Laboratorio: evoluciona el UML de captura a su versión 2

SITUACIÓN

Abre el UML v1 de la Clase anterior y actualiza su información interna.

1

**0–7 min: completa
visibilidad y tipos en
Entrenador, Inventario y
Equipo**

2

**7–14 min: escribe
parámetros y retornos
en al menos dos
métodos por clase**

3

**14–20 min: revisa
Equipo–Criatura y
Entrenador–Inventario**

4

**20–25 min: incorpora
una generalización de
Criatura solo si puedes
justificarla**

Producto: UML de captura v2 editable, con tres clases completas, dos relaciones revisadas, una generalización justificada y Uml_Pokemon_v2.md.

RELACIONES UML EN POO

Cómo se conectan las clases

RELACIONES ESTRUCTURALES



ASOCIACIÓN

Relación estable

Entrenador — Equipo



AGREGACIÓN

Parte que puede existir sola



Equipo ◇ Criatura



COMPOSICIÓN

Parte que depende del todo



Entrenador ◆ Inventario

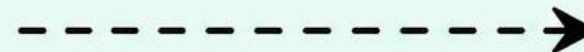


USO Y ESPECIALIZACIÓN



DEPENDENCIA

Uso temporal



Entrenador - - - > Criatura



HERENCIA

Es un tipo de



CriaturaFuego —> Criatura



Un buen UML comunica reglas, no solamente símbolos

TICKET DE SALIDA

¿Qué símbolo utilizarías para proteger un atributo, qué significa void y qué pregunta distingue agregación de composición?

PRÓXIMA CLASE

Clase 6: transferiremos el análisis al caso EcoTech y activaremos la EF1 de selección al finalizar la sesión.